Django Production Deployment – Complete Technical Guide

1. Clean Architecture Overview

```
Internet → Nginx (Reverse Proxy + SSL Termination) → Gunicorn (WSGI Server) → Django Applicati
```

Nginx handles HTTPS, static files, and reverse proxying. Gunicorn runs multiple worker processes and executes Django's WSGI application. Systemd supervises and auto-restarts Gunicorn. Django handles routing, business logic, templates, and database interactions.

2. Request Lifecycle (Step-by-Step)

1. Browser sends HTTPS request. 2. Nginx performs SSL handshake and decrypts traffic. 3. Nginx checks location blocks. 4. Static files served directly (if matched). 5. Otherwise, request is proxied to Gunicorn. 6. Gunicorn worker executes Django WSGI app. 7. Django processes middleware → URL routing → View → Template. 8. Response returns via Gunicorn → Nginx → Browser.

3. systemd Service File (Gunicorn Supervisor)

```
[Unit]
Description=Django App (Gunicorn)
After=network.target

[Service]
User=ubuntu
Group=www-data
WorkingDirectory=/path/to/project
Environment="PATH=/path/to/venv/bin"
ExecStart=/path/to/venv/bin/gunicorn --workers 3 --bind 127.0.0.1:9103 project.wsgi:application
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

• User/Group → Security isolation • WorkingDirectory → Execution context • Environment → Virtualenv activation • ExecStart → Starts Gunicorn with workers • Restart → Automatic crash recovery

4. Nginx Server Block Example

```
server {
    listen 443 ssl;
    server_name example.com;

    ssl_certificate /etc/letsencrypt/live/example.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/example.com/privkey.pem;

    location /static/ {
        alias /path/to/staticfiles/;
    }

    location / {
        proxy_pass http://127.0.0.1:9103;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto https;
    }
}
```

• listen → Port binding • ssl_certificate → TLS configuration • location → URL matching • alias → Static file offloading • proxy_pass → Reverse proxy forwarding

5. Gunicorn Worker & Memory Model

Gunicorn uses a master-worker architecture. Master Process: • Spawns worker processes • Monitors workers • Restarts crashed workers Workers: • Each handles one request at a time (sync model) • Each loads full Django app in memory Memory Usage Rule: Total RAM ≈ Worker Count × Memory per Worker

6. Production Best Practices

Security: • DEBUG=False • Set ALLOWED_HOSTS properly • Configure CSRF_TRUSTED_ORIGINS • Use HTTPS only • Enable firewall (ufw) Performance: • Use Nginx for static files • Enable gzip compression • Set proper worker count (2 × CPU + 1 rule) • Use caching if needed (Redis) Monitoring: • Use journalctl for service logs • Use Nginx access/error logs • Monitor disk usage (df -h) • Monitor memory (free -m) Scaling: • Vertical scaling → Upgrade instance • Horizontal scaling → Load balancer + multiple EC2 • Use RDS for production DB instead of SQLite

You now understand: • Reverse proxy architecture • TLS termination • WSGI execution model • Process supervision • Multi-worker concurrency • Static offloading • Production deployment lifecycle